I have a Pygame coin collector game. Bug: in update() in game/game_engine.py, check_collection is called every frame and a coin's value is added to the score for as long as the player overlaps it, because the coin is never removed after being collected. Fix it so each coin is collected exactly once, no matter how long the player touches or walks through it. Show me the full corrected code for each file you change, and explain in two lines what you changed. Here are my files:Coin: a static collectible circle.
Drawn as a circle, hit-tested as a
bounding square around it.
"""

```
import pygame


class Coin:
    def __init__(self, x, y, radius=12, value=1, color=(230, 190, 60)):
        self.x = x
        self.y = y
        self.radius = radius
        self.value = value
        self.color = color

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.radius), int(self.y - self.radius),
            self.radius * 2, self.radius * 2,
        ) this is the coin.py
```

The issue is that `coin.py` defines a coin but doesn't track whether it has already been collected. The cleanest fix is to remove collected coins from the game's active coin list in `game/game_engine.py`, so they cannot award points again.

1. `coin.py`

No changes are required to this file. Your existing code can stay exactly as it is:

Python

```
"""
Coin: a static collectible circle. Drawn as a circle, hit-tested as a
bounding square around it.
"""

import pygame


class Coin:
    def __init__(self, x, y, radius=12, value=1, color=(230, 190, 60)):
        self.x = x
        self.y = y
        self.radius = radius
        self.value = value
        self.color = color

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.radius), int(self.y - self.radius),
```

2. `game/game_engine.py`

I need your existing `game_engine.py` to provide the **full corrected file without changing unrelated game logic**. The fix should follow this pattern inside `update()`:

‹/› **Python**

```
# Check each active coin for collection.
for coin in self.coins[:]:
    if self.check_collection(coin):
        self.score += coin.value
        self.coins.remove(coin)
```

This example assumes `check_collection(coin)` returns `True` when the player collects that coin, and that your engine uses `self.coins` and `self.score`. If your current implementation differs, the code must be adapted to match it.

What changed (two lines):

1. Iterate over a copy of the active coin list and remove a coin immediately after collection.
2. Each coin can award its value only once, regardless of how long the player overlaps it.

Paste your `game/game_engine.py` next, and I'll return the complete corrected code for that file using your actual implementation.

"""
collection: figures out which coins the player has collected this frame.
"""

```
def check_collection(player, coins):
    """
    Returns the list of coins the player is currently overlapping.
    """
    player_rect = player.get_rect()
    collected = []
    for coin in coins:
        if player_rect.colliderect(coin.get_rect()):
            collected.append(coin)
    return collected this is collection.py and this is engine .py """
GameEngine: owns the player and all coins.

Starter version: one coin type, no obstacles, no timer yet. Coin collection also has a known bug (see how update uses check_collection
below) that Task 1 asks you to fix - collected coins are never removed,
so standing on one keeps awarding points every frame.
"""

import random
import pygame

from game.player import Player
from game.coin import Coin
from game.collection import check_collection
from game.renderer import WIDTH, HEIGHT

NUM_COINS = 6
COIN_VALUE = 1


class GameEngine:
```

```
    def __init__(self):
        self.player = Player(x=WIDTH / 2, y=HEIGHT / 2)
        self.coins = [self._random_coin() for _ in range(NUM_COINS)]
        self.score = 0

    def _random_coin(self):
        x = random.randint(30, WIDTH - 30)
        y = random.randint(30, HEIGHT - 30)
        return Coin(x=x, y=y, radius=12, value=COIN_VALUE)

    def handle_input(self, keys_pressed):
        dx = dy = 0
        if keys_pressed[pygame.K_UP]:
            dy -= self.player.speed
        if keys_pressed[pygame.K_DOWN]:
            dy += self.player.speed
        if keys_pressed[pygame.K_LEFT]:
            dx -= self.player.speed
        if keys_pressed[pygame.K_RIGHT]:
            dx += self.player.speed
        self.player.move(dx, dy, WIDTH, HEIGHT)

    def update(self):
        collected = check_collection(self.player, self.coins)
        for coin in collected:
            self.score += coin.value

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_scene(surface, self.player, self.coins)
        renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
```

You only need to change `game/game_engine.py`. The `collection.py` and `coin.py` files can remain unchanged.

1. `game/game_engine.py` — corrected code

‹/› **Python**

```
"""
GameEngine: owns the player and all coins.

Starter version: one coin type, no obstacles, no timer yet.
Collected coins are removed so each coin awards points only once.
"""

import random
import pygame

from game.player import Player
from game.coin import Coin
from game.collection import check_collection
from game.renderer import WIDTH, HEIGHT

NUM_COINS = 6
COIN_VALUE = 1
```

2. `game/collection.py`

No changes required. Your existing function correctly returns the list of coins currently overlapping the player.

What changed (two lines)

1. After adding a collected coin's value to the score, `self.coins.remove(coin)` removes it from the active coin list.
2. Future frames cannot collect that coin again because it is no longer in `self.coins`, even if the player stays in the same position.

the bug is still present. it keeps adding

If it's still adding points, the likely issue is that the collection logic is being triggered more than once, or the score is being updated somewhere else too. Let's make the collection logic robust by removing collected coins **before** awarding their points and ensuring we only process coins that are still active.

Replace `update()` in `game/game_engine.py` with this:

‹/› Python

```
def update(self):
    collected = check_collection(self.player, self.coins)
```

```
    for coin in collected:
        # Remove first so this coin cannot be collected again.
        if coin in self.coins:
            self.coins.remove(coin)
            self.score += coin.value
```

This guards against duplicate processing within the same update, but if the score still increases, I need to see where else it is updated.

Please search your project for `self.score +=` , `score +=` , **and** `check_collection(` . Also paste the code that calls `GameEngine.update()` (usually in your main game loop). There may be a second scoring path, or the engine may be recreated repeatedly.

pygame-ce 2.5.8 (SDL 2.32.10, Python 3.14.4)
Traceback (most recent call last):
File "C:\Users\DEVASHRUTH\Downloads\SE_vibecoding\06_coin_collector\coin-collector\main.py", line 11, in <module>
from game.game_engine import GameEngine
File "C:\Users\DEVASHRUTH\Downloads\SE_vibecoding\06_coin_collector\coin-collector\game\game_engine.py", line 45
collected = check_collection(self.player, self.coins)

The error points to line 45 in `game/game_engine.py` . It is most likely an **indentation or syntax error** from replacing only part of the `update()` method.

Replace your entire `game/game_engine.py` with this code. Make sure you copy everything, including the spaces before each indented line.

Python

```
"""
GameEngine: owns the player and all coins.

Collected coins are removed from the active list,
so each coin awards points only once.
"""

import random
import pygame
```

```
from game.player import Player
from game.coin import Coin
from game.collection import check_collection
from game.renderer import WIDTH, HEIGHT

NUM_COINS = 6
COIN_VALUE = 1
```

Why this fixes it

- **Syntax/indentation:** The complete class and `update()` method are correctly indented.
- **Duplicate collection:** Each coin is removed from `self.coins` before its value is added, so it cannot be collected again by subsequent calls to `update()`.

If the same error persists: Send me the full traceback, especially the final line that says `SyntaxError`, `IndentationError`, or another exception. That line identifies the exact cause.

Now Task 2: implement multiple coin types. Add at least three types: bronze (1 point), silver (3 points) and gold (5 points). Each type needs its own color so they look different, and the correct value must be added to the score when each one is collected. Spawn them randomly, with bronze being the most common and gold the rarest. Keep the collect-once fix I already have. Show me the full updated code for every file you change and explain what changed in two lines. Here are my current files:engine.py->

```
"""
GameEngine: owns the player and all coins.

Collected coins are removed from the active list,
so each coin awards points only once.
"""

import random
import pygame

from game.player import Player
from game.coin import Coin
from game.collection import check_collection
from game.renderer import WIDTH, HEIGHT

NUM_COINS = 6
```

```
COIN_VALUE = 1


class GameEngine:
    def __init__(self):
        self.player = Player(x=WIDTH / 2, y=HEIGHT / 2)
        self.coins = [self._random_coin() for _ in range(NUM_COINS)]
        self.score = 0

    def _random_coin(self):
        x = random.randint(30, WIDTH - 30)
        y = random.randint(30, HEIGHT - 30)
        return Coin(x=x, y=y, radius=12, value=COIN_VALUE)

    def handle_input(self, keys_pressed):
        dx = dy = 0

        if keys_pressed[pygame.K_UP]:
            dy -= self.player.speed
        if keys_pressed[pygame.K_DOWN]:
            dy += self.player.speed
        if keys_pressed[pygame.K_LEFT]:
            dx -= self.player.speed
        if keys_pressed[pygame.K_RIGHT]:
            dx += self.player.speed

        self.player.move(dx, dy, WIDTH, HEIGHT)

    def update(self):
        collected = check_collection(self.player, self.coins)

        for coin in collected:
            if coin in self.coins:
                self.coins.remove(coin)
                self.score += coin.value

    def draw(self, surface, font):
        from game import renderer

        renderer.draw_scene(surface, self.player, self.coins)
```

```
        renderer.draw_text(
            surface, font, f"Score: {self.score}", (10, 10)
        )  renderer.py ->"""
renderer: all pygame drawing lives here, kept separate from game
logic.
"""

import pygame

WIDTH, HEIGHT = 700, 500
WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (35, 45, 35)
COLOR_PLAYER = (80, 180, 255)
COLOR_TEXT = (255, 255, 255)


def draw_scene(surface, player, coins):
    surface.fill(COLOR_BG)
    for coin in coins:
        pygame.draw.circle(surface, coin.color, (int(coin.x), int(coin.y)),
coin.radius)
    pygame.draw.rect(surface, COLOR_PLAYER, player.get_rect(),
border_radius=4)


def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)


def draw_banner(surface, font, text):
    surf = font.render(text, True, (255, 220, 80))
    rect = surf.get_rect(center=(surface.get_width() // 2,
surface.get_height() // 2))
    surface.blit(surf, rect)  coin.py->"""
Coin: a static collectible circle. Drawn as a circle, hit-tested as a
bounding square around it.
"""

import pygame
```

```
class Coin:
    def __init__(self, x, y, radius=12, value=1, color=(230, 190, 60)):
        self.x = x
        self.y = y
        self.radius = radius
        self.value = value
        self.color = color

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.radius), int(self.y - self.radius),
            self.radius * 2, self.radius * 2,
        )
```

You only need to change `game/coin.py` and `game/game_engine.py`. Your existing `renderer.py` already draws coins using `coin.color`, so it will automatically display bronze, silver, and gold in different colors.

1. `game/coin.py` — updated code

Python

```python
"""
Coin: a collectible circle with a value and color.
Drawn as a circle, hit-tested as a bounding square around it.
"""

import pygame


class Coin:
    def __init__(self, x, y, radius=12, value=1, color=(205, 127, 50)):
        self.x = x
        self.y = y
        self.radius = radius
        self.value = value
        self.color = color

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.radius),
```

2. `game/game_engine.py` — updated code

Python

```python
"""
GameEngine: owns the player and all coins.

Collected coins are removed from the active list,
so each coin awards points only once.
"""

import random
import pygame

from game.player import Player
from game.coin import Coin
from game.collection import check_collection
from game.renderer import WIDTH, HEIGHT

NUM_COINS = 6

# Coin types: (value, color)
BRONZE = (1, (205, 127, 50))
```

3. `game/renderer.py`

No changes required. Your existing renderer uses `coin.color` when drawing each coin, so it supports all three types already.

What changed (two lines)

1. Added bronze (1 point, brown), silver (3 points, silver), and gold (5 points, yellow), with weighted random spawning at **60%, 30%, and 10%**, respectively.
2. Preserved the collect-once fix: each collected coin is removed from `self.coins` before its value is added to the score.

Now Task 3: add obstacles. Obstacles should appear in the play area and stay fully inside it. The player starts with 3 lives, and touching an obstacle costs one life. A single touch must not take a life every frame while the player is still overlapping the obstacle, so add a short invincibility period (about 1 second) after a hit, and make the player blink or change color during it. Obstacles must not spawn on top of the player or on top of coins. Show the lives

on screen next to the score. Keep the coin types and the collect-once fix working. Show me the full updated code for every file you change and explain what changed in two lines. Here are my current files:player.py ->"""
Player: a simple square the player moves around with the arrow keys.
"""

```
import pygame


class Player:
    def __init__(self, x, y, size=28, speed=4):
        self.x = x
        self.y = y
        self.size = size
        self.speed = speed

    def move(self, dx, dy, bounds_width, bounds_height):
        self.x = max(self.size / 2, min(bounds_width - self.size / 2,
self.x + dx))
        self.y = max(self.size / 2, min(bounds_height - self.size / 2,
self.y + dy))

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.size / 2), int(self.y - self.size / 2),
            self.size, self.size,
        ) the rest like coin renderer and engine are the ones u already
sent
```

For Task 3, I'll keep your bronze, silver, and gold coins, their weighted spawn probabilities, and the collect-once fix. I'll add rectangular obstacles, three starting lives, and a one-second invincibility window after each hit, with the player blinking during that window.

I'll use the existing `WIDTH` and `HEIGHT` boundaries so obstacles remain inside the play area, and check their positions against both the player and coins before spawning them.

1. `game/obstacle.py` — new file

This class represents a rectangular obstacle and provides the rectangle used for collision detection and drawing.

Python

```python
"""
Obstacle: a rectangular object that damages the player on contact.
"""

import pygame


class Obstacle:
    def __init__(self, x, y, width=50, height=30, color=(190, 65, 65)):
        self.rect = pygame.Rect(x, y, width, height)
        self.color = color

    def get_rect(self):
        return self.rect
```

2. `game/player.py` — updated code

The player now has an `invincible` flag that the engine updates after collisions. The renderer uses this flag to make the player blink.

Python

```python
"""
Player: a simple square the player moves around with the arrow keys.
"""

import pygame


class Player:
    def __init__(self, x, y, size=28, speed=4):
        self.x = x
        self.y = y
        self.size = size
        self.speed = speed
        self.invincible = False

    def move(self, dx, dy, bounds_width, bounds_height):
        self.x = max(
```

```
            self.size / 2,
```

3. `game/game_engine.py` — updated code

This version creates five obstacles, checks that they do not overlap the player, coins, or other obstacles, and subtracts a life on collision. The player receives 1,000 milliseconds of invincibility after being hit.

Python

```python
"""
GameEngine: owns the player, coins, and obstacles.

Features:
- Bronze, silver, and gold coins.
- Each coin awards points only once.
- Obstacles reduce lives on contact.
- One second of invincibility after an obstacle hit.
"""

import random
import pygame

from game.player import Player
from game.coin import Coin
from game.obstacle import Obstacle
from game.collection import check_collection
from game.renderer import WIDTH, HEIGHT
```

4. `game/renderer.py` — updated code

The renderer now draws obstacles and makes the player blink between colors during invincibility. The score and lives are displayed together by the engine.

Python

```python
"""
Renderer: all pygame drawing lives here, separate from game logic.
"""

import pygame
```

```
WIDTH, HEIGHT = 700, 500
WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (35, 45, 35)
COLOR_PLAYER = (80, 180, 255)
COLOR_PLAYER_HIT = (255, 90, 90)
COLOR_OBSTACLE = (190, 65, 65)
COLOR_TEXT = (255, 255, 255)


def draw_scene(surface, player, coins, obstacles):
    surface.fill(COLOR_BG)
```

What changed (two lines)

1. Added five randomly positioned obstacles that stay inside the play area and avoid the player, coins, and other obstacles; touching one costs a life, with three lives to start and a one-second invincibility period with blinking.
2. Updated rendering to show obstacles and display `Score` and `Lives` together, while preserving all three coin types and the collect-once scoring logic.

Note: If your project calls `renderer.draw_scene()` anywhere else, update that call to pass the obstacles list as the fourth argument.

```
PS
C:\Users\DEVASHRUTH\Downloads\SE_vibecoding\06_coin_collect
or\coin-collector> python main.py
pygame-ce 2.5.8 (SDL 2.32.10, Python 3.14.4)
Traceback (most recent call last):
  File
"C:\Users\DEVASHRUTH\Downloads\SE_vibecoding\06_coin_collec
tor\coin-collector\main.py", line 41, in <module>
    main()
    ~~~~^^
  File
"C:\Users\DEVASHRUTH\Downloads\SE_vibecoding\06_coin_collec
tor\coin-collector\main.py", line 32, in main
    engine.draw(screen, font)
    ~~~~~~~~~~~~^^^^^^^^^^^^^^^^
  File
"C:\Users\DEVASHRUTH\Downloads\SE_vibecoding\06_coin_collec
tor\coin-collector\game\game_engine.py", line 77, in draw
    renderer.draw_scene(surface, self.player, self.coins)
```

```
~~~~~~~~~~~~~~~~~~~~~^^^
```

```
python main.py
```

If another error appears, send me the traceback and we'll fix it without breaking the existing coin and obstacle logic.

TypeError: draw_scene() missing 1 required positional argument: 'obstacles'
PS C:\Users\DEVASHRUTH\Downloads\SE_vibecoding\06_coin_collector\coin-collector> python main.py
pygame-ce 2.5.8 (SDL 2.32.10, Python 3.14.4)
Traceback (most recent call last):
  File "C:\Users\DEVASHRUTH\Downloads\SE_vibecoding\06_coin_collector\coin-collector\main.py", line 11, in <module>
    from game.game_engine import GameEngine
  File "C:\Users\DEVASHRUTH\Downloads\SE_vibecoding\06_coin_collector\coin-collector\game\game_engine.py", line 147
    from game import renderer
    ^^^^
IndentationError: expected an indented block after function definition on line 146

You have an indentation error because the code inside `def draw(...)` is not indented correctly. Python expects every statement inside the method to be indented by four spaces relative to the method definition.

Fix it step by step

1. Open `game/game_engine.py` .
2. Go to the bottom of the file and replace your entire `draw()` method with this exact code. Make sure `def draw` has four spaces before it because it belongs inside the `GameEngine` class.

</> Python

```
    def draw(self, surface, font):
        from game import renderer

        renderer.draw_scene(
            surface,
```

```
            self.player,
            self.coins,
            self.obstacles,
        )

        renderer.draw_text(
            surface,
            font,
            f"Score: {self.score}   Lives: {self.lives}",
            (10, 10),
        )
```

3. Save with `Ctrl + S` and run:

powershell

```
python main.py
```

Important: The `def draw` line must have four spaces before it, and the method body must have eight spaces before each statement. Do not paste this method outside the `GameEngine` class.

If you still get an error, paste lines 130–160 of `game_engine.py` so I can identify the exact indentation problem.

give me the complete new engine code

Worked for 8s

Replace your entire `game/game_engine.py` with the code below. Select all the existing code using `Ctrl + A`, paste this, and save with `Ctrl + S`.

This version fixes the indentation problem and passes `self.obstacles` to `renderer.draw_scene()` correctly.

`game/game_engine.py`

Python

```
"""
GameEngine: owns the player, coins, and obstacles.

Features:
- Bronze, silver, and gold coins.
```

```
- Each coin awards points only once.
- Obstacles reduce player lives on contact.
- One second of invincibility after an obstacle hit.
"""

import random
import pygame

from game.player import Player
from game.coin import Coin
from game.obstacle import Obstacle
from game.collection import check_collection
from game.renderer import WIDTH, HEIGHT
```

Run the game

In your terminal, run:

</> powershell

```
python main.py
```

What changed (two lines):

1. Fixed the class and method indentation, and corrected the renderer call to include the obstacles list.
2. Preserved weighted bronze/silver/gold spawning, collect-once scoring, five obstacles, three lives, and one-second blinking invincibility after damage.

```
PS
C:\Users\DEVASHRUTH\Downloads\SE_vibecoding\06_coin_collect
or\coin-collector> python main.py
pygame-ce 2.5.8 (SDL 2.32.10, Python 3.14.4)
Traceback (most recent call last):
  File
"C:\Users\DEVASHRUTH\Downloads\SE_vibecoding\06_coin_collec
tor\coin-collector\main.py", line 11, in <module>
    from game.game_engine import GameEngine
  File
"C:\Users\DEVASHRUTH\Downloads\SE_vibecoding\06_coin_collec
tor\coin-collector\game\game_engine.py", line 17, in <module>
    from game.obstacle import Obstacle
```

ImportError: cannot import name 'Obstacle' from 'game.obstacle' (C:\Users\DEVASHRUTH\Downloads\SE_vibecoding\06_coin_collector\coin-collector\game\obstacle.py). Did you mean: 'obstacle'?
PS C:\Users\DEVASHRUTH\Downloads\SE_vibecoding\06_coin_collector\coin-co

The error means your `game/obstacle.py` file does not contain a class named `Obstacle`. Python even suggests that your class might currently be named `obstacle` with a lowercase `o`.

Fix: replace `game/obstacle.py`

Open `game/obstacle.py`, press `Ctrl + A`, and replace everything with this code:

Python

```python
"""
Obstacle: a rectangular object that damages the player on contact.
"""

import pygame


class Obstacle:
    def __init__(
        self,
        x,
        y,
        width=50,
        height=30,
        color=(190, 65, 65),
    ):
        self.rect = pygame.Rect(x, y, width, height)
        self.color = color
```

Then run the game again

1. Save the file with `Ctrl + S`.
2. Run this command from your project folder:

powershell

```
python main.py
```

Why this fixes it: Python is case-sensitive, so `Obstacle` and `obstacle` are different names. This defines the exact `Obstacle` class that your `game_engine.py` imports.

If another error appears, send me the complete traceback and we'll fix the next issue.

> the game is running well with the obstacles but i need the obstacles to be present around the coins everytime sometimes the obstacle are completely on the other side where there are no coins

The problem is that your current `_create_obstacles()` method places obstacles randomly anywhere in the play area. Let's change it so **each obstacle spawns near a coin**, making coins more challenging to collect, while keeping obstacles from overlapping coins or the player.

You only need to replace two methods in `game/game_engine.py`.

1. Replace `_create_obstacles()`

This version chooses a coin for each obstacle and tries to position the obstacle nearby. It also keeps obstacles inside the screen and avoids overlapping the player, coins, or other obstacles.

</> Python

```python
    def _create_obstacles(self, count):
        obstacles = []

        player_rect = self.player.get_rect().inflate(30, 30)

        for i in range(count):
            # Prefer placing each obstacle near a coin.
            for attempt in range(200):
                target_coin = self.coins[i % len(self.coins)]

                width = random.randint(40, 60)
                height = random.randint(20, 35)

                # Choose a position near the target coin.
                offset_x = random.randint(-90, 90)
                offset_y = random.randint(-90, 90)

                x = int(target_coin.x + offset_x - width / 2)
                y = int(target_coin.y + offset_y - height / 2)
```

2. Keep `__init__()` in this order

Make sure coins are created **before** obstacles, because obstacle placement now uses the coin positions.

Python

```
    def __init__(self):
        self.player = Player(x=WIDTH / 2, y=HEIGHT / 2)

        self.coins = [
            self._random_coin() for _ in range(NUM_COINS)
        ]

        self.obstacles = self._create_obstacles(NUM_OBSTACLES)

        self.score = 0
        self.lives = STARTING_LIVES
        self.invincible_until = 0
```

What changed (two lines)

1. Obstacles now target coin locations and spawn within roughly 90 pixels of a selected coin, instead of being distributed across the whole map.
2. Collision checks still prevent obstacles from overlapping coins, the starting player position, or one another, while keeping them fully inside the play area.

Note: Because the method retries up to 200 times per obstacle, a few obstacles may not be placed if no valid position is available near their target coin.

give me the complete engine code

Replace your entire `game/game_engine.py` with the code below. It keeps all your existing features and makes obstacles spawn **near coins instead of randomly across the map**.

Python

```
"""
GameEngine: owns the player, coins, and obstacles.

Features:
- Bronze, silver, and gold coins.
- Each coin awards points only once.
```

```
- Obstacles spawn near coins.
- Obstacles reduce player lives on contact.
- One second of invincibility after an obstacle hit.
"""

import random
import pygame

from game.player import Player
from game.coin import Coin
from game.obstacle import Obstacle
from game.collection import check_collection
from game.renderer import WIDTH, HEIGHT
```

What changed (two lines)

1. Obstacles now try to spawn within roughly 90 pixels of a selected coin, while avoiding coins, the player's starting position, other obstacles, and the screen boundaries.
2. All previous features remain: weighted bronze/silver/gold coins, collect-once scoring, three lives, collision damage, and one-second blinking invincibility.

Save the file and run `python main.py`. Note that if the game cannot find safe positions near coins after 200 attempts, it will create fewer than five obstacles rather than placing one on top of a coin or player.

> Now Task 4: add a timed round. Add a 30-second countdown and display the remaining time on screen. The round ends when the timer reaches zero or when lives reach zero, whichever comes first. When it ends, stop the gameplay (player, coins and obstacles stop updating) and show a game over screen with the final score clearly displayed and a message like 'Press R to play again'. When the player presses R, reset everything: score to 0, lives back to 3, timer back to 30 seconds, and respawn the coins and obstacles. Use delta time or the pygame clock for the timer so it counts real seconds, not frames. Keep everything from the earlier tasks working. Show me the full updated code for every file you change and explain what changed in two lines. Here are my current files:

Worked for 8s

We'll add the 30-second countdown while preserving your existing coin types, collect-once scoring, obstacles near coins, three lives, and one-second invincibility.

I need your current `main.py` to implement the restart correctly. The main loop handles keyboard events, so pressing `R` must be detected there; otherwise, the game-over screen might appear but

restarting won't work reliably.

Please paste `main.py`. I'll then give you the complete updated code for every file that needs changing, including:

- A real-time 30-second countdown.
- Game over when time reaches zero **or** lives reach zero.
- Frozen gameplay and a clear final-score screen.
- Press `R` to reset the score, lives, timer, coins, and obstacles for a fresh round.

```
"""
Coin Collector (Lab Starter)

Run with:  python3 main.py

Controls: Arrow keys to move.
"""

import pygame

from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE


def main():
    pygame.init()
    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Coin Collector")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont("consolas", 22)

    engine = GameEngine()
    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False

        keys = pygame.key.get_pressed()
        engine.handle_input(keys)
        engine.update()
```

```
    engine.draw(screen, font)

    pygame.display.flip()
    clock.tick(60)

pygame.quit()


if __name__ == "__main__":
    main()
```

We'll change `main.py` and `game/game_engine.py`. The engine will own the round timer and game-over state, while `main.py` will handle the `R` key to restart the round. The timer will use elapsed milliseconds from Pygame, so it won't depend on the frame rate.

1. `game/game_engine.py` — complete updated code

Python

```python
"""
GameEngine: owns the player, coins, and obstacles.

Features:
- Bronze, silver, and gold coins.
- Each coin awards points only once.
- Obstacles spawn near coins.
- Three lives and one-second invincibility after a hit.
- A 30-second timed round with a game-over state.
"""

import math
import random
import pygame

from game.player import Player
from game.coin import Coin
from game.obstacle import Obstacle
from game.collection import check_collection
```

2. `main.py` — complete updated code

Python

```
"""
Coin Collector

Run with: python main.py

Controls:
- Arrow keys: Move
- R: Restart after game over
- Close window: Quit
"""

import pygame

from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE


def main():
    pygame.init()
```

What changed (two lines)

1. Added a real-time 30-second countdown; the round ends when the timer reaches zero or lives reach zero, and player movement, coin collection, and collision updates stop.
2. Added a game-over overlay showing the final score and restart instructions; pressing **R** resets the timer, score, lives, player, coins, and obstacles for a new round.

Save both files and run `python main.py`.

> the press r to play again isnt working and the game sisnt restarting after collecting all the coins

There are two separate issues:

1. **Collecting all coins doesn't end the round:** the current engine only ends the game when time runs out or lives reach zero. We'll add a check for when `self.coins` becomes empty.
2. **Pressing R doesn't restart:** we'll make the restart handling explicit in `main.py`, so a fresh round is created only after game over.

The coin types, obstacle placement, collect-once scoring, and invincibility will remain intact.

1. Replace `game/game_engine.py`

This version ends the round when the timer expires, lives reach zero, **or all coins are collected**. The final coin's points are added before the game-over screen appears.

Python

```
"""
GameEngine: owns the player, coins, and obstacles.

Features:
- Bronze, silver, and gold coins.
- Each coin awards points only once.
- Obstacles spawn near coins.
- Three lives and one-second invincibility after a hit.
- 30-second round.
- Game over when time, lives, or coins run out.
"""

import math
import random
import pygame

from game.player import Player
from game.coin import Coin
from game.obstacle import Obstacle
```

2. Replace `main.py`

This version handles the `R` key using Pygame's keyboard event system. Restarting calls `engine.reset()`, which creates new coins and obstacles and resets all round values.

Python

```
"""
Coin Collector

Controls:
- Arrow keys: Move
- R: Restart after game over
- Close window: Quit
"""

import pygame

from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE


def main():
```

```
pygame.init()

screen = pygame.display.set_mode(WINDOW_SIZE)
```

What changed (two lines)

1. The round now ends when all coins are collected, the timer reaches zero, or lives reach zero; the final score remains visible.
2. Restarting with **R** calls `engine.reset()`, restoring three lives, 30 seconds, zero score, a new player, and freshly spawned coins and obstacles.

Test it: Collect all six coins. The game-over screen should appear immediately after the last coin is collected. Press `R` to start a fresh round.

thr r is working properly but i need the game to give me tht r window as soon as i finish collecting all the coins even if the timer isnt completed

The likely issue is that the game isn't detecting the empty coin list at the right point, or the last coin's collection isn't being processed as expected. Let's make the game-over condition explicit immediately after collecting coins.

In `game/game_engine.py`, replace your entire `update()` method with this version:

Python

```
def update(self):
    # Stop updating once the round has ended.
    if self.game_over:
        return

    # Update the real-time countdown.
    elapsed_ms = pygame.time.get_ticks() - self.start_ticks
    elapsed_seconds = elapsed_ms / 1000.0
    self.time_remaining = max(0, ROUND_DURATION - elapsed_seconds)

    if self.time_remaining <= 0:
        self.game_over = True
        return

    # Collect coins and remove them before adding points.
    collected = check_collection(self.player, self.coins)

    for coin in collected:
        if coin in self.coins:
```

Why this fixes it: The game checks `len(self.coins) == 0` immediately after processing collections, so the game-over overlay appears as soon as the final coin is removed, regardless of how much time remains. Your existing `main.py` already supports pressing **R** to restart.

give me the complete code

Replace your entire `game/game_engine.py` with the code below. It immediately shows the game-over screen when you collect the last coin, even if there is time remaining. Pressing **R** starts a fresh round using your existing `main.py`.

`game/game_engine.py` — complete updated code

‹/› Python

```python
"""
GameEngine: owns the player, coins, and obstacles.

Features:
- Bronze, silver, and gold coins.
- Each coin awards points only once.
- Obstacles spawn near coins.
- Three lives and one-second invincibility after a hit.
- 30-second countdown.
- Game over when time expires, lives reach zero,
  or all coins are collected.
"""

import math
import random
import pygame

from game.player import Player
from game.coin import Coin
```

What changed (two lines)

1. The engine checks `if not self.coins` immediately after awarding points, so collecting the final coin triggers game over without waiting for the 30-second timer.
2. The final score and **Press R to play again** message appear, and the existing restart handler resets the player, score, lives, timer, coins, and obstacles.

Save the file and run `python main.py`.